EasyLM: A Guide to Retrieval-Augmented Generation

EasyLM is a small web app that lets you chat with your own documents. You add PDFs, CSV files or pasted text, and then ask questions in plain language. Answers come from your documents instead of from whatever the language model happened to memorise during training.

The technique behind it is called retrieval-augmented generation, or RAG. A RAG system answers a question in two steps. First it retrieves the passages of your documents that are most relevant to the question. Then it generates an answer by giving those passages to a language model and asking it to answer using only that material.

This guide is the demo document bundled with EasyLM. Anything you ask about it is answered by the same pipeline that handles your own uploads, so you can see exactly how the product behaves.

1. How a document becomes searchable

When a file is added, EasyLM reads its text page by page. PDFs are parsed with pypdf, and CSV files are turned into one sentence-like record per row, so every row can be found on its own.

The text is then split into chunks of about 1000 characters with a 200 character overlap. The overlap keeps a sentence that straddles a boundary from being cut in half and losing its meaning.

Each chunk is converted into an embedding: a list of 384 numbers produced by the all-MiniLM-L6-v2 model. Chunks that mean similar things get similar numbers, even when they share no words. The embedding model runs locally on the server through ONNX, so no paid embedding service is needed.

Finally the vectors are stored in Qdrant, a vector database, together with the original text, the name of the source file and the workspace the chunk belongs to.

2. How a question is answered

When you ask a question, EasyLM embeds the question with the same model and asks Qdrant for the two chunks whose vectors are closest to it, using cosine similarity. Only chunks from the current workspace are considered, which is how one person's documents stay separate from another's.

The retrieved chunks are placed in the prompt of a large language model, which is hosted on Groq. The model used is openai/gpt-oss-120b with a low reasoning effort, which keeps replies fast. The prompt tells the model to answer briefly and only from the supplied context.

EasyLM remembers the conversation. A follow-up such as "and what about its battery?" is first rewritten into a standalone question using the last few turns, so the search finds the right chunks. The sources used are shown under each answer, so you can check where it came from.

3. Workspaces, accounts and guests

Anyone can open EasyLM as a guest. Guests use the shared demo workspace, which contains this guide. The demo is read-only: guests can ask questions and preview sources, but cannot upload or delete.

Creating a free account unlocks private workspaces. Each user can have up to five, and every workspace has its own address such as /u/yourname/workspace/1. Only the owner can see or change the documents in a workspace, and only the owner sees the delete buttons.

Passwords are hashed with scrypt and never stored in plain text. Sessions are signed tokens that expire after 30 days. Rate limits protect the free hosting: guests can ask 10 questions per minute, while logged-in users can ask 20 per minute.

4. Technology and limits

The frontend is built with React and Vite and is hosted on Vercel. The backend is a FastAPI application hosted on Render, with Qdrant Cloud as the database. Uploaded files are also kept in the browser, using IndexedDB, so the preview works even after a page reload.

Uploads can be PDF or CSV files up to 25 MB, or up to 200000 characters of pasted text. Scanned PDFs that contain only images of pages cannot be read, because EasyLM does not run OCR.

The free Groq plan allows roughly 8000 tokens per minute, which is why answers are kept short and only two chunks are retrieved per question. If a limit is reached, EasyLM shows a friendly message and asks you to retry in a moment.